JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Selector

java.lang.Object
    java.nio.channels.Selector

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

Direct Known Subclasses:

`AbstractSelector`

```
public abstract class Selector
extends Object
implements Closeable
```

A multiplexor of `SelectableChannel` objects.

A selector may be created by invoking the `open` method of this class, which will use the system's default `selector provider` to create a new selector. A selector may also be created by invoking the `openSelector` method of a custom selector provider. A selector remains open until it is closed via its `close` method.

A selectable channel's registration with a selector is represented by a `SelectionKey` object. A selector maintains three sets of selection keys:

- The *key set* contains the keys representing the current channel registrations of this selector. This set is returned by the `keys` method.
- The *selected-key set* is the set of keys such that each key's channel was detected to be ready for at least one of the operations identified in the key's interest set during a prior selection operation that adds keys or updates keys in the set. This set is returned by the `selectedKeys` method. The selected-key set is always a subset of the key set.
- The *cancelled-key* set is the set of keys that have been cancelled but whose channels have not yet been deregistered. This set is not directly accessible. The cancelled-key set is always a subset of the key set.

All three sets are empty in a newly-created selector.

A key is added to a selector's key set as a side effect of registering a channel via the channel's `register` method. Cancelled keys are removed from the key set during selection operations. The key set itself is not directly modifiable.

A key is added to its selector's cancelled-key set when it is cancelled, whether by closing its channel or by invoking its `cancel` method. Cancelling a key will cause its channel to be deregistered during the next selection operation, at which time the key will be removed from all of the selector's key sets.

Keys are added to the selected-key set by selection operations. A key may be removed directly from the selected-key set by invoking the set's `remove` method or by invoking the `remove` method of an `iterator` obtained from the set. All keys may be removed from the selected-key set by invoking the set's `clear` method. Keys may not be added directly to the selected-key set.

Selection

A selection operation queries the underlying operating system for an update as to the readiness of each registered channel to perform any of the operations identified by its key's interest set. There are two forms of selection operation:

1. The `select()`, `select(long)`, and `selectNow()` methods add the keys of channels ready to perform an operation to the selected-key set, or update the ready-operation set of keys already in the selected-key set.

2. The `select(Consumer)`, `select(Consumer, long)`, and `selectNow(Consumer)` methods perform an *action* on the key of each channel that is ready to perform an operation. These methods do not add to the selected-key set.

Selection operations that add to the selected-key set

During each selection operation, keys may be added to and removed from a selector's selected-key set and may be removed from its key and cancelled-key sets. Selection is performed by the `select()`, `select(long)`, and `selectNow()` methods, and involves three steps:

1. Each key in the cancelled-key set is removed from each key set of which it is a member, and its channel is deregistered. This step leaves the cancelled-key set empty.

2. The underlying operating system is queried for an update as to the readiness of each remaining channel to perform any of the operations identified by its key's interest set as of the moment that the selection operation began. For a channel that is ready for at least one such operation, one of the following two actions is performed:

    1. If the channel's key is not already in the selected-key set then it is added to that set and its ready-operation set is modified to identify exactly those operations for which the channel is now reported to be ready. Any readiness information previously recorded in the ready set is discarded.

    2. Otherwise the channel's key is already in the selected-key set, so its ready-operation set is modified to identify any new operations for which the channel is reported to be ready. Any readiness

information previously recorded in the ready set is preserved; in other words, the ready set returned by the underlying system is bitwise-disjoined into the key's current ready set.

If all of the keys in the key set at the start of this step have empty interest sets then neither the selected-key set nor any of the keys' ready-operation sets will be updated.

3. If any keys were added to the cancelled-key set while step (2) was in progress then they are processed as in step (1).

Whether or not a selection operation blocks to wait for one or more channels to become ready, and if so for how long, is the only essential difference between the three selection methods.

Selection operations that perform an action on selected keys

During each selection operation, keys may be removed from the selector's key, selected-key, and cancelled-key sets. Selection is performed by the `select(Consumer)`, `select(Consumer,long)`, and `selectNow(Consumer)` methods, and involves three steps:

1. Each key in the cancelled-key set is removed from each key set of which it is a member, and its channel is deregistered. This step leaves the cancelled-key set empty.

2. The underlying operating system is queried for an update as to the readiness of each remaining channel to perform any of the operations identified by its key's interest set as of the moment that the selection operation began.

   For a channel that is ready for at least one such operation, the ready-operation set of the channel's key is set to identify exactly the operations for which the channel is ready and the *action* specified to the `select` method is invoked to consume the channel's key. Any readiness information previously recorded in the ready set is discarded prior to invoking the *action*.

   Alternatively, where a channel is ready for more than one operation, the *action* may be invoked more than once with the channel's key and ready-operation set modified to a subset of the operations for which the channel is ready. Where the *action* is invoked more than once for the same key then its ready-operation set never contains operation bits that were contained in the set at previous calls to the *action* in the same selection operation.

3. If any keys were added to the cancelled-key set while step (2) was in progress then they are processed as in step (1).

Concurrency

A Selector and its key set are safe for use by multiple concurrent threads. Its selected-key set and cancelled-key set, however, are not.

The selection operations synchronize on the selector itself, on the selected-key set, in that order. They also synchronize on the cancelled-key set during steps (1) and (3) above.

Changes made to the interest sets of a selector's keys while a selection operation is in progress have no effect upon that operation; they will be seen by the next selection operation.

Keys may be cancelled and channels may be closed at any time. Hence the presence of a key in one or more of a selector's key sets does not imply that the key is valid or that its channel is open. Application code should be careful to synchronize and check these conditions as necessary if there is any possibility that another thread will cancel a key or close a channel.

A thread blocked in a selection operation may be interrupted by some other thread in one of three ways:

- By invoking the selector's `wakeup` method,
- By invoking the selector's `close` method, or
- By invoking the blocked thread's `interrupt` method, in which case its interrupted status will be set and the selector's `wakeup` method will be invoked.

The `close` method synchronizes on the selector and its selected-key set in the same order as in a selection operation.

A Selector's key set is safe for use by multiple concurrent threads. Retrieval operations from the key set do not generally block and so may overlap with new registrations that add to the set, or with the cancellation steps of selection operations that remove keys from the set. Iterators and spliterators return elements reflecting the state of the set at some point at or since the creation of the iterator/spliterator. They do not throw `ConcurrentModificationException`.

A selector's selected-key set is not, in general, safe for use by multiple concurrent threads. If such a thread might modify the set directly then access should be controlled by synchronizing on the set itself. The iterators returned by the set's `iterator` methods are *fail-fast:* If the set is modified after the iterator is created, in any way except by invoking the iterator's own `remove` method, then a `ConcurrentModificationException` will be thrown.

Since:

1.4

See Also:

SelectableChannel, SelectionKey

Constructor Summary

Constructors

Modifier	Constructor	Description
protected	Selector()	Initializes a new instance of this class.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
abstract void	close()	Closes this selector.
abstract boolean	isOpen()	Tells whether or not this selector is open.
abstract Set<SelectionKey>	keys()	Returns this selector's key set.
static Selector	open()	Opens a selector.
abstract SelectorProvider	provider()	Returns the provider that created this channel.
abstract int	select()	Selects a set of keys whose corresponding channels are ready for I/O operations.
abstract int	select(long timeout)	Selects a set of keys whose corresponding channels are ready for I/O operations.
int	select(Consumer<SelectionKey>	Selects and performs an action on the keys whose corresponding channels

		are ready for I/O operations.
int	**select**(Consumer<SelectionKey> long timeout)	Selects and performs an action on the keys whose corresponding channels are ready for I/O operations.
abstract Set<SelectionKey>	**selectedKeys**()	Returns this selector's selected-key set.
abstract int	**selectNow**()	Selects a set of keys whose corresponding channels are ready for I/O operations.
int	**selectNow**(Consumer<SelectionKey>	Selects and performs an action on the keys whose corresponding channels are ready for I/O operations.
abstract Selector	**wakeup**()	Causes the first selection operation that has not yet returned to return immediately.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

Selector

```
protected Selector()
```

Initializes a new instance of this class.

Method Details

open

```
public static Selector open()
                      throws IOException
```

Opens a selector.

The new selector is created by invoking the `openSelector` method of the system-wide default `SelectorProvider` object.

Returns:

A new selector

Throws:

`IOException` - If an I/O error occurs

isOpen

```
public abstract boolean isOpen()
```

Tells whether or not this selector is open.

Returns:

`true` if, and only if, this selector is open

provider

```
public abstract SelectorProvider provider()
```

Returns the provider that created this channel.

Returns:

The provider that created this channel

keys

```
public abstract Set<SelectionKey> keys()
```

Returns this selector's key set.

The key set is not directly modifiable. A key is removed only after it has been cancelled and its channel has been deregistered. Any attempt to modify the key set will cause an `UnsupportedOperationException` to be thrown.

The set is safe for use by multiple concurrent threads.

Returns:

This selector's key set

Throws:

`ClosedSelectorException` - If this selector is closed

selectedKeys

```
public abstract Set<SelectionKey> selectedKeys()
```

Returns this selector's selected-key set.

Keys may be removed from, but not directly added to, the selected-key set. Any attempt to add an object to the key set will cause an `UnsupportedOperationException` to be thrown.

The selected-key set is not thread-safe.

Returns:

This selector's selected-key set

Throws:

`ClosedSelectorException` - If this selector is closed

selectNow

```
public abstract int selectNow()
                        throws IOException
```

Selects a set of keys whose corresponding channels are ready for I/O operations.

This method performs a non-blocking selection operation. If no channels have become selectable since the previous selection operation then this method immediately returns zero.

Invoking this method clears the effect of any previous invocations of the `wakeup` method.

Returns:

The number of keys, possibly zero, whose ready-operation sets now indicate readiness for at least one category of operations for which the channel was not previously detected to be ready

Throws:

`IOException` - If an I/O error occurs

`ClosedSelectorException` - If this selector is closed

select

```
public abstract int select(long timeout)
                    throws IOException
```

Selects a set of keys whose corresponding channels are ready for I/O operations.

This method performs a blocking selection operation. It returns only after at least one channel is selected, this selector's `wakeup` method is invoked, the current thread is interrupted, or the given timeout period expires, whichever comes first.

This method does not offer real-time guarantees: It schedules the timeout as if by invoking the `Object.wait(long)` method.

Parameters:

`timeout` - If positive, block for up to `timeout` milliseconds, more or less, while waiting for a channel to become ready; if zero, block indefinitely; must not be negative

Returns:

The number of keys, possibly zero, whose ready-operation sets now indicate readiness for at least one category of operations for which the channel was not previously detected to be ready

Throws:

`IOException` - If an I/O error occurs

`ClosedSelectorException` - If this selector is closed

`IllegalArgumentException` - If the value of the timeout argument is negative

select

```
public abstract int select()
                   throws IOException
```

Selects a set of keys whose corresponding channels are ready for I/O operations.

This method performs a blocking selection operation. It returns only after at least one channel is selected, this selector's `wakeup` method is invoked, or the current thread is interrupted, whichever comes first.

Returns:

The number of keys, possibly zero, whose ready-operation sets now indicate readiness for at least one category of operations for which the channel was not previously detected to be ready

Throws:

`IOException` - If an I/O error occurs

`ClosedSelectorException` - If this selector is closed

select

```
public int select(Consumer<SelectionKey> action,
                  long timeout)
           throws IOException
```

Selects and performs an action on the keys whose corresponding channels are ready for I/O operations.

This method performs a blocking selection operation. It wakes up from querying the operating system only when at least one channel is selected, this selector's `wakeup` method is invoked, the current thread is interrupted, or the given timeout period expires, whichever comes first.

The specified *action*'s `accept` method is invoked with the key for each channel that is ready to perform an operation identified by its key's interest set. The `accept` method may be invoked more than once for the same key but with the ready-operation set containing a subset of the operations for which the channel is ready (as described above). The `accept` method is invoked while synchronized on the selector and its selected-key set. Great care must be taken to avoid deadlocking with other threads that also synchronize on these objects. Selection operations are not reentrant in general and consequently the *action* should take great care not to attempt a selection operation on the same selector. The behavior when attempting a reentrant selection operation is implementation specific and therefore not specified. If the *action* closes the selector then `ClosedSelectorException` is thrown when the action completes. The *action* is not prohibited from closing channels registered with the selector, nor prohibited from cancelling keys or changing a key's interest set. If a channel is selected but its key is cancelled or its interest set changed before the *action* is performed on the key then it is implementation specific as to whether the *action* is invoked (it may be invoked with an `invalid` key). Exceptions thrown by the action are relayed to the caller.

This method does not offer real-time guarantees: It schedules the timeout as if by invoking the `Object.wait(long)` method.

Implementation Requirements:

The default implementation removes all keys from the selected-key set, invokes `select(long)` with the given timeout and then performs the action for each key added to the selected-key set. The default implementation does not detect the action performing a reentrant selection operation. The selected-key set may or may not be empty on completion of the default implementation.

Parameters:

`action` - The action to perform

`timeout` - If positive, block for up to `timeout` milliseconds, more or less, while waiting for a channel to become ready; if zero, block indefinitely; must not be negative

Returns:

The number of unique keys consumed, possibly zero

Throws:

`IOException` - If an I/O error occurs

`ClosedSelectorException` - If this selector is closed or is closed by the action

`IllegalArgumentException` - If the value of the timeout argument is negative

Since:

11

select

```
public int select(Consumer<SelectionKey> action)
           throws IOException
```

Selects and performs an action on the keys whose corresponding channels are ready for I/O operations.

This method performs a blocking selection operation. It wakes up from querying the operating system only when at least one channel is selected, this selector's `wakeup` method is invoked, or the current thread is interrupted, whichever comes first.

This method is equivalent to invoking the 2-arg `select` method with a timeout of `0` to block indefinitely.

Implementation Requirements:

The default implementation invokes the 2-arg `select` method with a timeout of `0`.

Parameters:

`action` - The action to perform

Returns:

The number of unique keys consumed, possibly zero

Throws:

`IOException` - If an I/O error occurs

`ClosedSelectorException` - If this selector is closed or is closed by the action

Since:

11

selectNow

```
public int selectNow(Consumer<SelectionKey> action)
              throws IOException
```

Selects and performs an action on the keys whose corresponding channels are ready for I/O operations.

This method performs a non-blocking selection operation.

Invoking this method clears the effect of any previous invocations of the `wakeup` method.

Implementation Requirements:

The default implementation removes all keys from the selected-key set, invokes `selectNow()` and then performs the action for each key added to the selected-key set. The default implementation does not detect the action performing a reentrant selection operation. The selected-key set may or may not be empty on completion of the default implementation.

Parameters:

`action` - The action to perform

Returns:

The number of unique keys consumed, possibly zero

Throws:

`IOException` - If an I/O error occurs

`ClosedSelectorException` - If this selector is closed or is closed by the action

Since:

11

wakeup

```
public abstract Selector wakeup()
```

Causes the first selection operation that has not yet returned to return immediately.

If another thread is currently blocked in a selection operation then that invocation will return immediately. If no selection operation is currently in progress then the next invocation of a selection operation will return immediately unless `selectNow()` or `selectNow(Consumer)` is invoked in the meantime. In any case the value returned by that invocation may be non-zero.

Subsequent selection operations will block as usual unless this method is invoked again in the meantime.

Invoking this method more than once between two successive selection operations has the same effect as invoking it just once.

Returns:

This selector

close

```
public abstract void close()
                    throws IOException
```

Closes this selector.

If a thread is currently blocked in one of this selector's selection methods then it is interrupted as if by invoking the selector's `wakeup` method.

Any uncancelled keys still associated with this selector are invalidated, their channels are deregistered, and any other resources associated with this selector are released.

If this selector is already closed then invoking this method has no effect.

After a selector is closed, any further attempt to use it, except by invoking this method or the `wakeup` method, will cause a `ClosedSelectorException` to be thrown.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Throws:

`IOException` - If an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.